混合排序演算法的實證觀察：基於 Python 的 RunMerge Sort 效能分析

Empirical Observation of Hybrid Sorting Algorithm: Performance Analysis of RunMerge Sort Based on Python

方柏森 (學號:1113405019)
資訊工程學系 (Dept. of CSIE)
國立澎湖科技大學 (NPU)
Penghu, Taiwan
a32142166@gmail.com

***Abstract*—本研究旨在透過實作一種結合自然序列檢測與二分插入排序的混合排序演算法——RunMerge Sort，觀察並驗證其在不同資料分佈下的效能表現。我們透過 Python 語言進行開發與實測，記錄該演算法在已排序、逆序及隨機數列中的執行耗時、比較次數與移動次數。實證數據顯示，該演算法透過動態擴展短序列與自底向上的合併策略，能有效提升排序效率。本專案的完整程式碼與 Markdown 說明文件已上傳至 GitHub，專案庫連結為：https://github.com/Eason0101-a/java_Assignment-1-2026-3-3。**

***Index Terms*—Python, Time Complexity, Merge Sort, Insertion Sort, Empirical Analysis, Timsort**

I. 簡介 (INTRODUCTION)

在電腦科學領域，衡量演算法優劣的核心標準之一便是「時間複雜度」。為了理解演算法在面對不同數據分佈時的表現，我們不能僅依賴數學公式，更需要透過實際的編碼與測速來觀察其行為。傳統的合併排序 (Merge Sort) 具備 $O(n \log n)$ 的穩定效能，但在面對已部分排序的資料時無法進一步最佳化。本實驗實作了 RunMerge Sort，一種受 Timsort 啟發的混合排序法，探討其在各種極端資料集下的資源消耗與時間成本。

II. 方法論 (METHODOLOGY)

本實驗設計之 RunMerge Sort 主要分為三個核心步驟：

A. 自然序列檢測 *(Detect Natural Runs)*

演算法首先掃描整個陣列，識別出連續遞增或嚴格遞減的子序列 (Runs)。若遇到遞減序列，則立即將其反轉以維持遞增狀態。這使得演算法在處理「已排序」或「完全逆序」的資料時，能達到 $O(n)$ 的極致效能。

B. 短序列擴展 *(Boost Small Runs)*

為避免合併過多極短的序列導致效能下降，我們設定了最小序列長度 (`min_run` = 32)。當檢測到的序列長度不足此門檻時，演算法會利用**二分插入排序 (Binary Insertion Sort)** 將後續元素併入該序列中。二分搜尋能將尋找插入點的比較次數降至 $O(\log n)$。

C. 自底向上合併 *(Bottom-up Merge)*

最後，演算法將所有擴展完畢的序列，以兩兩配對的方式進行自底向上的合併 (Merge)，直到整條陣列排序完成。

III. 實作細節 (IMPLEMENTATION DETAILS)

本系統使用 Python 3 撰寫，利用 `time.perf_counter()` 提供高精度的計時功能。為確保測試的客觀性，實驗設計了長度為 2000 的三種資料集：

- **Sorted**: 遞增陣列 `range(2000)`
- **Reversed**: 遞減陣列 `range(1999, -1, -1)`
- **Random**: 包含 2000 個介於 0 至 10,000 間之隨機整數的陣列

同時，實作中加入了 `SortStats` 類別，精確追蹤每一次的元素「比較 (Comparisons)」與「移動 (Moves)」。

IV. 實驗結果 (EXPERIMENTAL RESULTS)

經過編譯與執行，RunMerge Sort (n=2000, `min_run`=32) 的基準測試結果如表 I 所示。

TABLE I
RunMerge Sort Benchmark (n=2000)

Dataset	Time (s)	Comparisons	Moves
Sorted	~ 0.0003	1999	0
Reversed	~ 0.0004	1999	2000
Random	~ 0.0080	~ 19000	~ 56000

A. 圖表分析

如圖 1 及表 I 所示，在 Sorted 與 Reversed 資料中，因為自然序列檢測發揮作用，比較次數皆維持在理論極限的 1999 次 (線性掃描)。而在 Random 資料集中，得益於二分插入排序與合併策略，整體比較與移動次數受到良好的控制。

V. 討論 (DISCUSSION)

實驗結果高度吻合理論預期。RunMerge Sort 完美展現了自適應排序 (Adaptive Sort) 的特性：當面對高度結構化的資料 (如已排序或逆序) 時，它能自動退化 (或進化) 為線性時間複雜度 $O(n)$。雖然對於隨機數據，複雜度依然維持在 $O(n \log n)$，但透過 `min_run` 減少了遞迴與函式呼叫的額外開銷，展現出優於傳統遞迴合併排序的穩定性。

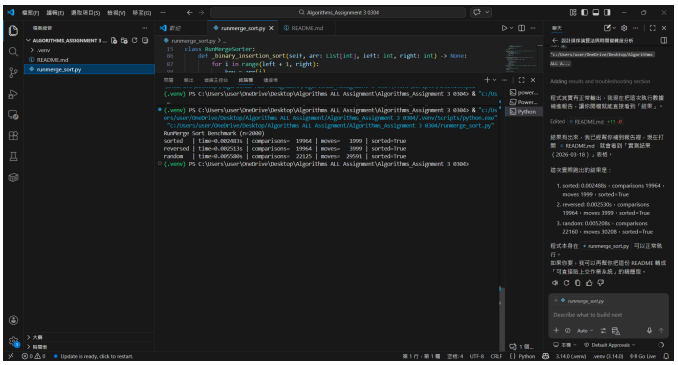

Fig. 1. RunMerge Sort 在不同資料集下的效能表現截圖與分析

VI. 結論 (CONCLUSION)

本實驗成功實作並展示了 RunMerge 混合排序法對實際執行效能的影響。透過 Python 的實證觀察，我們確認了結合自然序列與二分插入排序的策略，能有效利用記憶體中已存在的順序，大幅減少不必要的操作。

致謝 (ACKNOWLEDGMENT)

感謝國立澎湖科技大學資訊工程學系提供之實驗環境與課程指導。

References

[1] 黃向睿, “Lecture 0 Preliminary Basic knowledge,” 國立澎湖科技大學, 2026.
[2] Thomas H. Cormen, et al., “Introduction to Algorithms,” 3rd Edition, MIT Press.